



STUDY/COURSE GUIDE

COS 102: Problem Solving





Course Material Development Team

Content Editor: Dr Emeka Ogbuju, PhD, Artificial Intelligence

Course Editor: Dr Joshua Osah, PhD, Information Systems

Course Coordinator: Chioma Isiekwene, MSc Computer Science

Course Information

Course Code: COS 102

Course Title: Problem-Solving

Credit Unit: 3

Course Status: C

Learning Hours (LH): 30

Practical Hours (PH): 45

Ice Breaker

While modern computers are incredibly powerful, their ability to solve any problem is entirely dependent on the clarity and logic of the instructions they receive from humans. The core challenge of computing, therefore, is not the machine's processing power but the human skill of breaking down complex, often ambiguous, problems into a precise sequence of executable steps. This course is designed to equip you with that fundamental skill—the art and science of computational thinking and problem-solving—which forms the bedrock of all programming and software development.

Contents

Introduction	2
Course Aims	3
Course Objectives	3
Working through this Course	4
What you will learn in the Course	4
Course Materials	5
Study Sessions	5
Set of Textbooks/References	7
Assignment File	7
Glossary	8
Presentation Schedule	8
Course Marking Scheme	8
Course Overview	9
How to get the best from the Course	10
Assessment	10
Tutor-Marked Assignments (TMAs)	10
Final Examination and Grading	11
Tutors and Tutorials	11
Summary	11

Introduction

COS 102 is a foundational one-semester course that provides an intensive study of Problem-Solving in computing. It focuses on the core concepts of computing problems, the processes involved in solving them, and the techniques for designing and representing solutions. The course is designed to transition you from simply using software to understanding how to instruct a computer to solve problems logically and efficiently. You will learn to think like a computer scientist, a skill that is critical for all subsequent programming and computer science courses.

Course Aims

This course is designed to:

- Introduce you to the fundamental principles of computational thinking and problem-solving.
- Equip you with a toolkit of various techniques and methods for tackling different types of computing problems.
- Enable you to formally represent your solution designs using standard tools like algorithms, flowcharts, and pseudocode.
- Provide a foundational understanding of how these designed solutions are implemented using programming languages like C and Python.
- Foster a mindset of evaluation and refinement for developing optimal solutions.

Course Objectives

By the end of this course, you will be able to:

1. Explain the problem-solving processes and differentiate between routine and non-routine problems.
2. Demonstrate proficient problem-solving skills applicable to computing scenarios.
3. Describe the concept of algorithm development and the essential properties of a good algorithm.
4. Discuss and apply various solution techniques such as abstraction, analogy, brainstorming, and divide-and-conquer.
5. Solve computer problems by designing solutions using algorithms, flowcharts, and pseudocode.
6. Solve basic problems by implementing algorithms in a programming language such as C or Python.

Working through this Course

There are twenty-one study units which you have to go through in this course. You should study the contents in each unit before you attempt the questions provided after each content. Furthermore, you should have your writing materials (pen, paper, and other necessary materials) readily available throughout your weeks of learning this course because there are lots of learning activities that will require you to take notes in order to get the best out of the learning experience, as this will reinforce learning. You should pay attention to the objectives of each study unit to guide you through the unit. You should be ready to think and write as you go through this course material because it has been designed to make you do so. You will be assessed through tutor-marked assignments which you are expected to do and turn in to your tutor at the right time. You are also expected to write an examination or seminar papers or both at the end of the course. The time of the examination or the time to begin writing seminar paper will be communicated to you.

What you will learn in the Course

In COS 102, you will embark on a comprehensive journey into the heart of computing. You will start by understanding what constitutes a "problem" in a computational context, learning to identify and classify different problem types. You will then explore a wide array of systematic techniques for devising solutions, moving from abstract ideas to concrete, structured designs. A significant portion of the course will be dedicated to mastering the representation of these solutions through algorithms, flowcharts, and pseudocode—the universal blueprints of software. Finally, you will translate these blueprints into executable code using foundational programming languages, thereby completing the full problem-solving cycle from identification to implementation, evaluation, and refinement.

Course Materials

The major components of the course are:

1. Course guide
2. Study units
3. Recommended textbooks and resources
4. Assignments
5. Presentation Schedule

Study Sessions

The course consists of 12 study units, aligned with 12 weeks of study sessions:

Study Session 1: Introduction to Core Computing Concepts and Problems

- Overview of computing; definition of a problem; introduction to problem-solving.

Study Session 2: Types of Computing Problems

- Routine vs. non-routine problems; solvable vs. unsolvable problems.

Study Session 3: The Problem-Solving Process I

- General overview of the problem-solving process: identification, understanding, and planning.

Study Session 4: The Problem-Solving Process II

- General overview continued: implementation, evaluation, and refinement.

Study Session 5: Solution Techniques I

- Techniques: Abstraction, Analogy, Brainstorming, Trial and Error.

Study Session 6: Solution Techniques II

- Techniques: Hypothesis Testing, Reduction, Divide and Conquer, Literal Thinking.

Study Session 7: Solution Techniques III

- Techniques: Means-End Analysis, Method of Focal Object, Morphological Analysis, Research, Root Cause Analysis.

Study Session 8: Introduction to Algorithms

- Definition and concept of algorithms; properties of a good algorithm.

Study Session 9: Solution Formulation and Design I

- Designing solutions using Flowcharts and Pseudocode.

Study Session 10: Solution Formulation and Design II

- Designing solutions using Decision Tables and Decision Trees.

Study Session 11: Implementation Fundamentals

- Introduction to basic programming concepts (variables, data types, operators) in C/Python.

Study Session 12: Implementing Solutions

- Writing simple programs to solve problems using control structures (sequence, selection, iteration) in C/Python.

Set of Textbooks/References

1. **How to Solve It: A New Aspect of Mathematical Method** by George Pólya
 - **Relevance:** A classic text that provides a universal framework for problem-solving, applicable far beyond mathematics to computing.
2. **Introduction to Algorithms** by **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein**
 - **Relevance:** A comprehensive reference on algorithm design, analysis, and implementation. Essential for understanding the properties and development of algorithms.
3. **Think Like a Programmer: An Introduction to Creative Problem-Solving** by V. Anton Spraul
 - **Relevance:** Focuses directly on the problem-solving techniques used by programmers, including breaking down problems and overcoming obstacles.

Assignment File

In this file you will find all the details of the work you must submit to your tutor for grading. The marks you obtain from these assignments will form part of your final course mark. Additional information on assignments will be found in the assignment file itself as well as in the section on assessment in this course.

Glossary

- Algorithm: A step-by-step procedure or formula for solving a problem in a finite number of steps.
- Flowchart: A graphical representation of an algorithm, using symbols to represent different types of actions or steps.
- Pseudocode: An informal, high-level description of the operating principle of a computer program or algorithm that uses the structural conventions of programming languages but is intended for human reading.
- Abstraction: The process of hiding unnecessary details to focus on the essential characteristics of a problem.
- Divide and Conquer: A problem-solving technique that breaks a problem into smaller, more manageable sub-problems, solves them independently, and combines their solutions.

Presentation Schedule

The "presentation schedule" which has been included in your course materials gives you the important dates you are expected to complete your Tutor-Marked Assignments and attend tutorials. Remember, you are required to submit your assignments whenever it is expected of you.

Course Marking Scheme

The table below gives a breakdown of the Course Mark

Assessment	Marks
Continuous Assessment (CA)	CA1 (20%) + CA2 (20%) = 40% of total course marks.
Final examination	The final examination counts for 60% of overall marks
Total	100% of course marks

Table 1: *Course Marking Scheme*

Course Overview

Units	Title of Work	Week's Activity	Assessment (end of unit)
1	Core Concepts of Computing	1	Assignment 1
2	Problems and Problem-Solving	2	Assignment 2
3	Identification and Types of Problems	3	Assignment 3
4	Introduction to Algorithms and Heuristics	4	Assignment 4

5	Solvable and Unsolvable Problems	5	Assignment 5
6	General Problem-Solving Process	6	Assignment 6
7	Abstraction, Analogy, and Brainstorming	7	Assignment 7
8	Trial and Error, Hypothesis Testing, and Reduction	8	Assignment 8
9	Other Techniques (Lateral Thinking, Means-End Analysis, etc.)	9	Assignment 9
10	Flowcharts and Pseudocode	10	Assignment 10
11	Decision Tables and Decision Trees	11	Assignment 11
12	Implementation and Programming in C/Python	12	Assignment 12

How to get the best from the Course

The study units in this course have been written in such a way that you will understand them without the lecturer being physically there with you: This is why your programme is a Distance Learning one. Each study unit is designed for a week's work. The study unit will introduce you to the topic meant for the week; it will give you the stated/expected objectives for the unit and what you are expected to be able to do at the end of the unit. You only need to be focused and consistent in your effort to mastering the critical areas of computational problem-solving. If you follow the

instructions and do the exercises that follow, you will find yourself proficient in designing and implementing solutions to a wide array of computing problems.

Assessment

You will be assessed in three ways in this course, (1) through the tutor-marked assignment, (2) through the computer-marked assessment and (3) through written examination. You are expected to do the assignments and submit them to your tutorial facilitator for formal assessment in accordance with the stated deadlines in the presentation schedule and the 'assignment' file. Your tutor-marked assignments and computer-marked assessment will both account for 40% of the total course mark.

Tutor-Marked Assignments (TMAs)

Continuous assessment which includes the assignments will be 40% of your total course mark. Assignments for the units in this course are contained in the assignment file. You should be able to complete your assignments from the information materials contained in your set of textbooks, reading and study units. You are expected to use other references to broaden your knowledge of the subject.

Final Examination and Grading

The final examination for COS 102 will be a three-hour paper. The pattern of questions will test both theoretical understanding and practical problem-solving skills. The sixty marks for the final examination and the forty marks for the continuous assessment will give you a total of one hundred marks (i.e. $60 + 40 = 100$).

Tutors and Tutorials

The course includes 15 hours of tutorials, which will be delivered through a combination of recorded videos (asynchronous learning), live sessions (synchronous

learning), office hours, group work, and projects. Your tutorial group will be communicated to you. Feel free to relate with your tutorial facilitator who will mark, correct your assignments and monitor your attendance/performance in tutorial-marked assignments. You should always contact your tutorial facilitator by e-mail if you have any problem with the contents of any of the study units.

Summary

COS 102 is specifically designed to build your foundational competence in computational problem-solving. You will learn to systematically approach problems, apply a variety of techniques to devise solutions, and represent these solutions clearly using standard tools. The course culminates in the practical application of these skills through basic programming, providing a crucial bridge between theoretical problem-solving and practical implementation. This knowledge is essential for your success in all future computer science and programming courses.